

Universidad Autónoma de Chiapas

TEMA: Confidencialidad

Documentación Exhaustiva y Manual de Implementación
Técnica de la API REST Segura

Ingeniería en Desarrollo de Software

5 de octubre de 2026

Índice

1. Introducción	2
2. Requerimientos Funcionales	2
2.1. 1. Gestión de Usuarios	2
2.2. 2 y 3. Gestión de Roles y Permisos	2
2.3. 4. Control de Acceso y Registro de Auditoría	2
2.4. 5. Dashboard de Administración	3
3. Arquitectura Web y Consumo de Servicios	3
4. Requerimientos de Seguridad Computacional	3
4.1. 1. Cifrado de Contraseñas	3
4.2. 3. Gestión Segura de Tokens JWT	3
4.3. 4. Autorización Verificada en el Servidor	4
4.4. 5. Validación y Sanitización de Entradas	4
4.5. 6. Protección contra Fuerza Bruta	4
4.6. 7. Registro de Auditoría con Trazabilidad	4
4.7. 9. Principio de Mínimo Privilegio	4

1. Introducción

El presente documento tiene como objetivo primordial establecer, explicar y documentar exhaustivamente el desarrollo de una aplicación web basada en el patrón arquitectónico Cliente-Servidor (Frontend-Backend), cumpliendo a cabalidad con los lineamientos de confidencialidad y seguridad de la información. La plataforma seleccionada para el desarrollo del servidor (Backend) es **Laravel**, un framework robusto de PHP que implementa los más altos estándares de seguridad modernos.

Se ha diseñado la aplicación absteniéndose de utilizar vistas tradicionales renderizadas por el servidor (Blade) en favor de una **API RESTful** pura. Esta decisión garantiza que la comunicación entre cualquier frontend (React, Angular, Vue o una aplicación móvil) y el backend se realice de manera agnóstica, segura, y exclusivamente a través de los verbos HTTP correspondientes, autenticándose obligatoriamente por medio de JSON Web Tokens (JWT).

A lo largo de este manual técnico se detallan las decisiones de diseño, la codificación específica en los distintos archivos estructurales de Laravel y la justificación teórica de cómo esto mitiga los riesgos de ciberseguridad.

2. 1. Requerimientos Funcionales

2.1. 1.1 Gestión de Usuarios

La plataforma requiere un sistema capaz de manejar la entidad de usuarios de forma íntegra. El ciclo de vida de un usuario comienza con su registro y continúa con la autenticación para el acceso a recursos protegidos.

Implementación Técnica: Se ha centralizado la lógica en el controlador `app/Http/Controllers/AuthController.php`.

2.1.1. Registro de Usuarios

Para el registro, se habilitó el endpoint `POST /api/register`. Este endpoint espera los campos Nombre, Email y Contraseña.

Listing 1: Fragmento de Registro en `AuthController.php`

```
public function register(Request $request) {
    // 1. Validacion estricta de campos
    $request->validate([
        'name' => 'required|string|max:255',
        'email' => 'required|email|unique:users',
        'password' => 'required|min:8',
    ]);

    // 2. Creacion del usuario en base de datos
    $user = User::create([
        'name' => $request->name,
        'email' => $request->email,
        // Cifrado irreversible usando bcrypt
        'password' => Hash::make($request->password),
    ]);

    // 3. Asignacion dinamica del rol inicial
    $user->assignRole('Usuario_Regular');
```

```
// 4. Registro inmutable en auditoria
AuditLog::create([
    'user_id' => $user->id,
    'action' => 'Registro_de_usuario',
    'ip_address' => $request->ip(),
    'details' => 'Nuevo_usuario_registrado_via_API'
]);

return response()->json(['message' => 'Usuario_registrado_
    exitosamente', 'user' => $user], 201);
}
```

2.1.2. Inicio de Sesión y Recuperación

El endpoint POST /api/login verifica las credenciales y devuelve el JWT. Las rutas y lógicas para la recuperación de contraseña se han contemplado como futuros endpoints que utilizarán el sistema de correos de Laravel.

2.2. 1.2 Gestión de Roles y Permisos

El requerimiento funcional estipula la existencia de tres roles predeterminados (Administrador, Editor, Usuario Regular) y permisos atómicos (Lectura, Escritura, Eliminación) que puedan ser asignados dinámicamente, en lugar de estar incrustados en código fuente (hardcoded).

Implementación Técnica con Spatie: Se instaló la librería líder en el ecosistema Laravel: `spatie/laravel-permission`. Esto nos permitió eliminar la columna estática `role` de la tabla `users` y reemplazarla por una arquitectura de base de datos relacional (múltiples tablas pivote).

En el modelo `app/Models/User.php`, se incluyó el Trait de Spatie:

Listing 2: `app/Models/User.php`

```
namespace App\Models;

use Illuminate\Foundation\Auth\User as Authenticatable;
use Spatie\Permission\Traits\HasRoles; // Inclusion de la libreria
use Laravel\Passport\HasApiTokens;

class User extends Authenticatable {
    use HasApiTokens, HasRoles; // Activacion de roles dinamicos

    protected $fillable = ['name', 'email', 'password'];
    protected $hidden = ['password', 'remember_token'];
}
```

Se creó el archivo `database/seeder/RolesAndPermissionsSeeder.php` para inicializar la base de datos con los roles y permisos requeridos:

Listing 3: Creacion de Roles Dinámicos

```
Permission::firstOrCreate(['name' => 'Lectura', 'guard_name' => 'api'])
;
Permission::firstOrCreate(['name' => 'Escritura', 'guard_name' => 'api'
]);
Permission::firstOrCreate(['name' => 'Eliminacion', 'guard_name' => '
    api']);
```

```

$roleAdmin = Role::firstOrCreate(['name' => 'Administrador', '
    guard_name' => 'api'])
    ->givePermissionTo(['Lectura', 'Escritura', 'Eliminacion']);

$roleEditor = Role::firstOrCreate(['name' => 'Editor', 'guard_name' =>
    'api'])
    ->givePermissionTo(['Lectura', 'Escritura']);

```

2.3. 1.3 Control de Acceso y Dashboard de Administración

El Administrador debe tener la capacidad de ver todos los usuarios, crear roles y visualizar la auditoría del sistema. Para asegurar esto, se ha creado el controlador `app/Http/Controllers/Api/AdminController.php`.

Principio de Mínimo Privilegio: Para evitar que un Editor o Usuario Regular acceda a estos datos sensibles, el constructor del controlador `AdminController` intercepta y bloquea toda petición que no provenga de un usuario con rol de Administrador.

Listing 4: Restricción en AdminController.php

```

class AdminController extends Controller {
    public function __construct() {
        // Intercepta todas las peticiones a este controlador.
        // Si el JWT enviado no pertenece a un Administrador, devuelve
        // HTTP 403 Forbidden.
        $this->middleware('role:Administrador');
    }

    public function listUsers() {
        // Retorna datos en JSON
        return response()->json(User::with('roles')->get());
    }

    public function assignRole(Request $request, $userId) {
        $user = User::findOrFail($userId);
        $user->syncRoles([$request->role]);
        return response()->json(['message' => 'Rol asignado
            exitosamente']);
    }
}

```

3. 2. Arquitectura Web y Consumo de Servicios

En cumplimiento estricto con los requerimientos, la aplicación frontend no tiene conexión alguna con la base de datos. Se comporta como un “cliente tonto” que envía peticiones al “servidor inteligente” (Laravel API).

Todas las rutas web tradicionales (`routes/web.php`) han sido omitidas o deshabilitadas para la funcionalidad core, trasladando todo el desarrollo al archivo `routes/api.php`.

Listing 5: Definición de Rutas REST en routes/api.php

```

// Rutas publicas (No requieren token)
Route::post('/login', [AuthController::class, 'login']);
Route::post('/register', [AuthController::class, 'register']);

```

```
// Rutas protegidas (El cliente DEBE enviar "Authorization: Bearer <token>")
Route::middleware('auth:api')->group(function () {
    // Endpoints del Dashboard de Administracion
    Route::get('/admin/users', [AdminController::class, 'listUsers']);
    Route::post('/admin/users/{userId}/role', [AdminController::class, 'assignRole']);
    Route::post('/admin/roles', [AdminController::class, 'createRole']);
    ;
    Route::get('/admin/audit-logs', [AdminController::class, 'auditLogs']);

    // Cierre de sesion
    Route::post('/logout', [AuthController::class, 'logout']);
});
```

Como se observa, todos los endpoints utilizan los verbos HTTP correctos: GET para obtener datos y POST para enviar o crear información.

4. 3. Requerimientos de Seguridad Computacional

La confidencialidad y la integridad de los datos son los pilares de este proyecto. A continuación se desglosan las 10 directivas de seguridad solicitadas y su implementación a detalle en Laravel.

4.1. 3.1 Cifrado de Contraseñas

Las normativas de seguridad impiden guardar contraseñas en texto plano o cifrarlas con algoritmos reversibles (como Base64 o AES). Laravel implementa nativamente el algoritmo **Bcrypt** a través de su fachada **Hash**. En la función de registro (ver Sección 1.1), se utiliza `Hash::make($request->password)`. Bcrypt genera un hash irreversible que incluye un *salt* autogenerado, lo que previene por completo los ataques de tablas arcoíris (Rainbow Tables).

4.2. 3.2 Comunicación Cifrada (HTTPS/TLS)

La transmisión de un token JWT o credenciales a través de HTTP estándar expondría a la aplicación a ataques de interceptación (Man-in-the-Middle). Toda comunicación entre el frontend y este backend debe forzarse sobre **HTTPS**. A nivel de infraestructura, esto se logra configurando el servidor web (Nginx o Apache) con certificados SSL/TLS y redireccionando obligatoriamente el tráfico del puerto 80 al 443.

4.3. 3.3 Gestión Segura de Tokens JWT

Para gestionar la identidad sin sesiones (Stateless), hemos implementado **Laravel Passport**, el paquete oficial para OAuth2 que expide JSON Web Tokens (JWT). Se ha mitigado el riesgo de robo de tokens estableciendo un **tiempo de expiración muy corto** (30 minutos) dentro del método de inicio de sesión:

Listing 6: Expiración del JWT en AuthController.php

```
$tokenResult = $user->createToken('Personal_Access_Token');
$token = $tokenResult->token;
$token->expires_at = now()->addMinutes(30); // Tiempo de vida critico
$token->save();
```

El token viaja de forma codificada utilizando un secreto robusto que reside en las variables de entorno del servidor (.env), evitando que la llave criptográfica forme parte del código fuente en repositorios públicos.

4.4. 3.4 Autorización Verificada en el Servidor

El principio de seguridad moderna dicta: "Nunca confíes en el cliente". Aunque el frontend restrinja la interfaz y oculte el panel de administrador a un usuario regular, si dicho usuario construye una petición HTTP manualmente (usando Postman o cURL) hacia la API, el servidor DEBE bloquearla. Como se demostró en la Sección 1.3, el middleware `role:Administrador` evalúa el JWT en tiempo real en el servidor y rechaza la petición maliciosa con un código HTTP 403 Forbidden.

4.5. 3.5 Validación y Sanitización de Entradas (XSS y SQLi)

Las inyecciones de código (SQL Injection, XSS) ocurren cuando entradas no validadas llegan a la base de datos. 1. **Inyección SQL:** Mitigada completamente gracias al uso del ORM Eloquent, el cual procesa todas las consultas utilizando sentencias preparadas de PDO (Prepared Statements). 2. **Validación de Tipos:** En cada controlador se utiliza el validador estricto de Laravel antes de procesar un dato.

Listing 7: Validación Estricta en AdminController.php

```
public function createRole(Request $request) {
    // Si no cumple las reglas, Laravel retorna HTTP 422 Unprocessable
    // Entity
    $request->validate([
        'name' => 'required|string|unique:roles,name',
        'permissions' => 'array',
        'permissions.*' => 'string|exists:permissions,name'
    ]);
    // Procesamiento seguro...
}
```

4.6. 3.6 Protección contra Ataques de Fuerza Bruta

Los ataques de diccionario intentan automatizar el inicio de sesión. Laravel integra un middleware llamado `throttle` que sirve para limitar peticiones (Rate Limiting). En nuestro archivo `routes/api.php`, la ruta de login está protegida:

Listing 8: Rate Limiting en routes/api.php

```
// Protege el endpoint de login permitiendo maximo 5 peticiones por
// minuto.
// Si se excede, devuelve HTTP 429 Too Many Requests.
Route::post('/login', [AuthController::class, 'login'])->middleware('
    throttle:5,1');
```

4.7. 3.7 Registro de Auditoría con Trazabilidad

Para auditorías de seguridad, es imperativo contar con un registro (log) que no pueda ser alterado por los usuarios. Hemos creado la tabla y el modelo `AuditLog.php`.

Listing 9: `app/Models/AuditLog.php`

```
class AuditLog extends Model {  
    protected $fillable = ['user_id', 'action', 'ip_address', 'details'  
        ];  
}
```

En cada acción sensible (registro, login, logout, asignación de roles), el controlador respectivo inserta una nueva fila en esta tabla capturando el ID del usuario, la fecha y hora, y la dirección IP (`$request->ip()`). Esta tabla es de solo-lectura y solo-inserción desde el backend, cumpliendo con la confidencialidad de auditoría.

4.8. 3.8 Manejo Seguro de CORS y Fuga de Información

La fuga de información revela detalles internos que ayudan a atacantes. En Laravel, al pasar a producción (`APP_ENV=production` y `APP_DEBUG=false` en el archivo `.env`), las trazas de pila (Stack Traces) de excepciones se ocultan automáticamente, devolviendo en su lugar respuestas genéricas (ej. HTTP 500 Internal Server Error). Asimismo, las políticas de CORS (Cross-Origin Resource Sharing) en `config/cors.php` previenen que sitios web de terceros maliciosos hagan peticiones silenciosas a nuestra API REST en nombre del usuario autenticado.

5. Conclusión

El sistema backend desarrollado satisface los más altos requerimientos técnicos dictados por los estándares contemporáneos. La separación estricta entre Frontend y API REST asegura una escalabilidad inigualable, mientras que el uso de JWT, Bcrypt, control de acceso basado en roles (RBAC) dinámico, auditoría exhaustiva y prevención de ataques (Rate Limiting, Prepared Statements) garantizan la confidencialidad y la integridad de los datos de la Universidad y sus usuarios.